

Carbon Accounting App - Functional Repository Structure

🎯 Overview

This document outlines how to restructure your repository by **functionality** rather than technical layers, making debugging and feature development much more efficient.

📁 Proposed Folder Structure

```
carbon-accounting-app/  
├── 🗝️ 01-authentication/      # All auth-related functionality  
├── 📝 02-data-input/          # Emission data entry and management  
├── 📊 03-analytics-monitoring/ # Charts, reports, and data visualization  
├── 🏠 04-administration/      # Admin panel, user management, system control  
├── 🏗️ 05-infrastructure/      # Database, middleware, configurations  
├── 🎨 06-ui-components/       # Reusable UI components and styling  
├── 🔧 07-data-services/        # API services, utilities, data management  
├── 📱 08-mobile-responsive/    # Mobile-specific components and styles  
├── 🧪 09-testing/             # All test files and testing utilities  
└── 📖 10-documentation/       # Docs, guides, and API documentation
```

🗝️ 01-authentication/

Purpose: Handle all user authentication, authorization, and session management

Backend Files:

```
01-authentication/  
├── backend/  
│   ├── controllers/  
│   │   └── authController.js  
│   ├── middleware/  
│   │   └── auth.js  
│   ├── models/  
│   │   └── User.js (extracted from models/index.js)  
│   ├── routes/  
│   │   └── auth.js  
│   └── utils/  
│       └── emailService.js
```

Frontend Files:

```
├── frontend/
```

```
| |—— components/
| | |—— ProtectedRoute.jsx
| | |—— AdminRoute.jsx
| |—— context/
| | |—— AuthContext.jsx
| |—— pages/
| | |—— Login.jsx
| | |—— Register.jsx
| |—— services/
| |—— authAPI.js (extracted from api.js)
```

When to debug here: Login issues, permission errors, token problems, registration failures



02-data-input/

Purpose: All functionality related to inputting and managing emission data

Backend Files:

```
02-data-input/
|—— backend/
| |—— controllers/
| | |—— emissionController.js
| |—— middleware/
| | |—— validation.js
| |—— models/
| | |—— Emission.js (extracted from models/index.js)
| |—— routes/
| |—— emissions.js
```

Frontend Files:

```
|—— frontend/
| |—— components/
| | |—— EmissionForm/
| | |—— InfoTooltip/
| |—— data/
| | |—— emissionFactors.js
| |—— pages/
| | |—— Input/
| |—— utils/
| |—— localStorage.js
| |—— sampleDataSeeder.js
| |—— validation.js
```

When to debug here: Form submission errors, data validation issues, emission calculations, storage problems

03-analytics-monitoring/

Purpose: Data visualization, reporting, analytics, and monitoring functionality

Backend Files:

```
03-analytics-monitoring/
├── backend/
│   ├── controllers/
│   │   ├── analyticsController.js
│   │   ├── monitorController.js
│   │   └── exportController.js
│   ├── models/
│   │   ├── Activity.js (extracted from models/index.js)
│   │   └── Task.js (extracted from models/index.js)
│   └── routes/
│       ├── analytics.js
│       ├── monitor.js
│       └── export.js
```

Frontend Files:

```
├── frontend/
│   ├── components/
│   │   ├── ActivityTable/
│   │   ├── AddTaskModal/
│   │   └── Pagination/
│   ├── context/
│   │   └── ActivityContext.jsx
│   ├── pages/
│   │   ├── Analytics/
│   │   ├── Monitor/
│   │   └── Dashboard/
│   └── utils/
│       ├── formatters.js
│       └── constants.js
```

When to debug here: Chart rendering issues, data aggregation problems, export failures, dashboard loading errors

👑 04-administration/

Purpose: Admin-only functionality including user management and system monitoring

Backend Files:

```
04-administration/  
├── backend/  
│   ├── controllers/  
│   │   ├── adminController.js  
│   │   └── userController.js  
│   └── routes/  
│       ├── admin.js  
│       └── users.js
```

Frontend Files:

```
├── frontend/  
│   ├── components/  
│   │   └── [Admin-specific components]  
│   ├── pages/  
│   │   ├── Admin/  
│   │   └── Permissions/  
│   └── services/  
│       └── adminAPI.js (extracted from api.js)
```

When to debug here: User management issues, admin panel errors, permission problems, system monitoring failures

🏗️ 05-infrastructure/

Purpose: Core system infrastructure, database, middleware, and configurations

Backend Files:

```
05-infrastructure/  
├── backend/  
│   ├── config/  
│   │   └── database.js  
│   ├── middleware/  
│   │   └── errorHandler.js  
│   ├── models/  
│   │   ├── index.js (remaining models)  
│   │   └── Vehicle.js
```

```
| | |—— Generator.js
| | |—— Organization.js
| | |—— Notification.js
| |—— utils/
| | |—— seeder.js
| |—— server.js
| |—— package.json
| |—— .env
```

Frontend Files:

```
|—— frontend/
| |—— layouts/
| | |—— Layout.jsx
| |—— main.jsx
| |—— App.jsx
| |—— index.html
| |—— package.json
| |—— vite.config.js
| |—— tailwind.config.js
| |—— postcss.config.js
| |—— .env
```

When to debug here: Server startup issues, database connection problems, build errors, environment configuration

🎨 06-ui-components/

Purpose: Reusable UI components, styling, and layout components

Frontend Files:

```
06-ui-components/
|—— components/
| |—— Header/
| |—— Sidebar/
| |—— PageHeader/
| |—— NotificationCard/
|—— styles/
| |—— index.css
|—— assets/
| |—— [images, icons, etc.]
```

When to debug here: UI rendering issues, styling problems, component behavior errors, responsive design issues

07-data-services/

Purpose: API communication, data transformation, and service utilities

Backend Files:

```
07-data-services/  
├── backend/  
│   ├── controllers/  
│   │   ├── dashboardController.js  
│   │   ├── vehicleController.js  
│   │   ├── generatorController.js  
│   │   ├── organizationController.js  
│   │   └── notificationController.js  
│   └── routes/  
│       ├── dashboard.js  
│       ├── vehicles.js  
│       ├── generators.js  
│       ├── organization.js  
│       └── notifications.js
```

Frontend Files:

```
├── frontend/  
│   ├── context/  
│   │   └── NotificationContext.jsx  
│   ├── services/  
│   │   └── api.js  
│   ├── pages/  
│   └── Settings/
```

When to debug here: API communication failures, data transformation errors, service integration issues

09-testing/

Purpose: All test files, test utilities, and testing configurations

09-testing/

```
|—— backend/
| |—— tests/
| | |—— auth.test.js
| | |—— emissions.test.js
| | |—— admin.test.js
| |—— test-utils/
|—— frontend/
| |—— tests/
| | |—— components/
| | |—— pages/
| | |—— services/
| |—— test-utils/
|—— e2e/
|—— [end-to-end tests]
```



10-documentation/

Purpose: Documentation, API guides, and development resources

10-documentation/

```
|—— api/
| |—— authentication.md
| |—— emissions.md
| |—— admin.md
|—— frontend/
| |—— components.md
| |—— state-management.md
|—— deployment/
|—— SETUP.md
|—— CONTRIBUTING.md
|—— DEBUGGING.md
```



Migration Strategy

Step 1: Create New Folder Structure

bash

```
mkdir -p 01-authentication/{backend,frontend}
mkdir -p 02-data-input/{backend,frontend}
mkdir -p 03-analytics-monitoring/{backend,frontend}
mkdir -p 04-administration/{backend,frontend}
mkdir -p 05-infrastructure/{backend,frontend}
mkdir -p 06-ui-components
mkdir -p 07-data-services/{backend,frontend}
mkdir -p 09-testing/{backend,frontend,e2e}
mkdir -p 10-documentation/{api,frontend,deployment}
```

Step 2: Move Files Systematically

Start with one functional area at a time:

1. **Authentication first** (critical path)
2. **Infrastructure second** (dependencies)
3. **Data Input** (core feature)
4. **Analytics & Monitoring**
5. **Administration**
6. **UI Components**
7. **Data Services**

Step 3: Update Import Paths

Update all import statements to reflect new paths:

```
javascript

// Before
import { authAPI } from '../services/api';

// After
import { authAPI } from '../01-authentication/frontend/services/authAPI';
```

Step 4: Update Build Configurations

Modify `vite.config.js` and other build tools to recognize new structure.

Debugging Benefits

Quick Problem Location

- **Authentication issues** → Check `01-authentication/`

- **Form not working** → Check `02-data-input/`
- **Charts broken** → Check `03-analytics-monitoring/`
- **Admin panel errors** → Check `04-administration/`
- **Server won't start** → Check `05-infrastructure/`

Parallel Development

- Multiple developers can work on different functional areas without conflicts
- Clear ownership boundaries
- Easier code reviews focused on specific functionality

Testing Strategy

- Unit tests co-located with functionality
- Integration tests for cross-functional features
- End-to-end tests in dedicated folder

 Best Practices

1. **Keep related files together** - If debugging emission calculations, everything needed is in `02-data-input/`
2. **Clear naming conventions** - Folder numbers indicate dependency order
3. **Shared utilities** - Common utilities go in `07-data-services/`
4. **Documentation per feature** - Each folder can have its own README
5. **Environment separation** - Backend and frontend clearly separated within each functional area

 Common Debugging Scenarios

Issue Type	Primary Folder	Secondary Folders
Login failing	<code>01-authentication/</code>	<code>05-infrastructure/</code>
Data not saving	<code>02-data-input/</code>	<code>07-data-services/</code>
Charts not loading	<code>03-analytics-monitoring/</code>	<code>07-data-services/</code>
Permissions error	<code>04-administration/</code>	<code>01-authentication/</code>
Build failing	<code>05-infrastructure/</code>	-
UI broken	<code>06-ui-components/</code>	<code>03-analytics-monitoring/</code>
API not responding	<code>07-data-services/</code>	<code>05-infrastructure/</code>

This structure transforms your debugging process from "hunting through the entire codebase" to "checking the specific functional area" - dramatically reducing time to resolution and improving code maintainability.